Production App Rebuild Playbook

A repeatable workflow for reverse-engineering a real production app and independently rebuilding it as a cohesive, production-grade system.

Core Mindset

Do **not** rebuild screen-by-screen.

Think:

Screen = projection of state
State = result of workflows
Workflow = business capability
Capability = domain rules + data
Data = authority + persistence + synchronization
Everything = constrained by lifecycle, failure, security, concurrency, and performance

The goal is **behavioral and engineering equivalence**, not reproducing the original company's source code or invisible infrastructure.

Master Flow

> ***Choose → Explore → Specify → Model → Architect → Set up → Implement → Integrate → Harden → Release***

Repeat this loop for every reference app.

Phase 1 — Reference App Selection

Goal

Define exactly what you are going to rebuild.

Do

- Choose a real production app.

- Explore its major capabilities.
- Estimate complexity.
- Decide platform and technology scope.
- Decide which backend/services you will build, mock, or replace.
- Define what “production-grade” means for this project.
- Explicitly define what is in and out of scope.

Questions

- What are the major business capabilities?
- Which capabilities are essential?
- Which capabilities can be excluded?
- Is the app complex enough to teach useful system design?
- What parts require a backend?
- What parts require persistence, synchronization, notifications, authentication, etc.?

Deliverables

```
docs/
└── planning/
    ├── scope.md
    ├── feature-scope.md
    ├── success-criteria.md
    └── constraints.md
```

Phase 2 — Reference App Exploration / Reverse Engineering

Goal

Understand **what the app does** before deciding how to build it.

Do not design Android classes or modules yet.

Explore

For every feature:

- Main user journey
- Alternate journeys

- Navigation
- Loading states
- Empty states
- Error states
- Retry/recovery
- Refresh behavior
- Persistence
- Permissions
- Background behavior
- Notifications
- Authentication/session behavior
- Restart/process-death behavior
- Offline/network-loss behavior where relevant

For Every Important Action Ask

1. What triggers it?
2. What data is required?
3. What changes?
4. What other features are affected?
5. What happens if it fails?
6. What happens if the user leaves the screen?
7. What happens after app restart?
8. What happens if the network disappears?
9. What happens if the same action happens twice?
10. What happens if responses arrive out of order?

Example

Instead of simply documenting:

> *Cart screen has quantity buttons.*

Investigate:

- How is cart state created?
- Where is it persisted?
- Is it server-owned or locally owned?
- Does quantity update immediately?
- What happens when the network fails?
- Can quantity changes race?

- Does the cart survive restart?
- Does checkout use the same cart state?
- What happens after login/logout?

Deliverables

```
docs/
└── discovery/
    ├── feature-inventory.md
    ├── screen-inventory.md
    ├── user-journeys.md
    ├── behavior-observations.md
    ├── error-scenarios.md
    ├── lifecycle-observations.md
    └── unknowns.md
```

Phase 3 — Requirements & Workflows

Goal

Convert observations into an implementable specification.

Group screens and behaviors into **business capabilities**, not UI screens.

Do

- Define functional requirements.
- Define main and alternate workflows.
- Identify business rules.
- Identify validations.
- Identify state transitions.
- Define failure behavior.
- Separate confirmed facts from assumptions.
- Define acceptance criteria.

Important Rule

At this point, requirements should be explainable **without mentioning Android classes, ViewModels, repositories, Compose, or modules**.

Example:

Bad:

> `CartViewModel` updates `CartRepository`.

Good:

> *When a user changes item quantity, the cart quantity changes, the total is recalculated, and the updated cart state remains consistent with checkout.*

Deliverables

```
docs/
└── requirements/
    ├── functional-requirements.md
    ├── business-rules.md
    ├── workflows/
    ├── state-machines/
    ├── acceptance-criteria.md
    └── assumptions-and-unknowns.md
```

Phase 4 — Domain & System Modeling

Goal

Understand the underlying system structure.

Identify

- Entities
- Value objects
- Relationships
- Business rules
- Invariants
- State machines
- Capability ownership
- Data ownership

- Source of truth
- Dependencies
- Communication boundaries

For Every Important Entity Ask

- Who owns it?
- Who can modify it?
- What is the source of truth?
- What rules constrain it?
- What states can it have?
- What transitions are valid?
- What happens when an operation fails?
- What happens when two operations happen concurrently?

Deliverables

```
docs/
└── design/
    ├── domain-model.md
    ├── entity-relationships.md
    ├── data-ownership.md
    ├── invariants.md
    ├── state-machines/
    └── system-context.md
```

Phase 5 — Architecture & Technology Design

Goal

Turn the system model into an implementation structure.

A reasonable starting conceptual structure is:

```
App / Composition
        ↓
Presentation
        ↓
Domain
        ↓
Data
        ↓
Infrastructure
```

App / Composition

- Application startup
- Dependency injection
- Navigation composition
- Feature composition

Presentation

- Screens
- UI state
- ViewModels/presentation logic
- User interaction handling

Domain

- Business rules
- Entities/value objects
- Use cases
- Domain contracts

Data

- Repositories
- API clients
- Local database
- Cache
- Mapping
- Synchronization

Infrastructure

- Network
- Storage
- Security

- Platform services
- External services

Design

- Capability boundaries
- Dependency direction
- Module boundaries
- Contracts
- Persistence strategy
- Cache strategy
- Synchronization strategy
- Error strategy
- Authentication/session strategy
- Navigation strategy
- Security strategy
- Testing strategy

Important Rule

Do not create architecture just because it looks “production-grade”.

Every abstraction and boundary should solve a real problem.

Deliverables

```
docs/
└── architecture/
    ├── system-architecture.md
    ├── module-structure.md
    ├── dependency-graph.md
    ├── data-flow.md
    ├── error-strategy.md
    ├── security-design.md
    └── adr/
```

Phase 6 — Project Foundation

Goal

Build a reliable technical foundation before implementing many features.

Set Up

- Android project
- Gradle configuration
- Kotlin/Compose if chosen
- Dependency management
- Dependency injection
- Networking
- Serialization
- Database/local storage if required
- Secure storage if required
- Navigation foundation
- Structured logging
- Error handling
- Unit testing
- Integration testing
- Lint/static analysis
- Formatting
- CI
- Automated builds

Possible Structure

```
app/

core/
├── common/
├── network/
├── database/
├── security/
├── logging/
└── testing/

domain/
├── auth/
├── catalog/
├── cart/
├── checkout/
└── orders/

data/
├── auth/
├── catalog/
├── cart/
├── checkout/
└── orders/

feature/
├── auth/
├── catalog/
├── product/
├── cart/
├── checkout/
└── orders/
```

This is an example, **not a rigid template**.

Foundation Success Criteria

The project should:

- Compile.
- Run.
- Navigate through basic structure.
- Execute automated checks.
- Have working infrastructure.

- Have a clear dependency direction.
- Be ready for the first vertical slice.

Do **not** implement every feature yet.

Phase 7 — Incremental Feature Implementation / Vertical Slices

Goal

Build complete capabilities end-to-end.

Do not build:

> *all UI → all ViewModels → all repositories → all APIs → integration later*

Instead build:

> *one capability → complete → verify → integrate → move on*

For Each Capability

1. Select the capability

Prioritize based on:

- Dependencies
- Architectural risk
- Technical uncertainty
- Business importance

2. Finalize requirements

Know:

- Behavior
- Business rules
- States
- Failure behavior
- Acceptance criteria

3. Implement domain

- Models
- Invariants
- Business rules
- Use cases
- Contracts

4. Implement data

- API contracts
- Repositories
- Persistence
- Cache
- Synchronization where necessary

5. Implement presentation

- UI state
- ViewModels/presentation logic
- Screens
- Navigation

6. Integrate

Make sure the capability uses existing state ownership correctly.

Avoid creating duplicate sources of truth.

7. Test

Test:

- Business rules
- Data behavior
- UI states
- Failure cases
- Lifecycle behavior
- Integration behavior

8. Harden

Before moving on:

- Remove hacks.
- Fix architectural problems.
- Refactor duplication.
- Verify boundaries.

- Update documentation.

Example Dependency Order

For a commerce application:

```
Auth
  ↓
Catalog
  ↓
Search
  ↓
Product
  ↓
Cart
  ↓
Address
  ↓
Checkout / Payment
  ↓
Orders
  ↓
Profile / Settings
```

The exact order should be determined by your system’s dependencies and risks.

Phase 8 — Integration & Whole-System Validation

Goal

Make sure individually working features also work together.

Test Full Journeys

Examples:

```
Login
  → Browse
  → Search
  → Product
  → Add to Cart
  → Change Quantity
  → Checkout
  → Place Order
  → View Order
```

Validate

- Shared state consistency
- Data ownership
- Navigation
- Deep links
- Cross-feature interactions
- Session expiration
- Session restoration
- Logout
- Persistence
- App restart
- Process death
- Concurrent operations
- Out-of-order responses
- Network failures

Deliverables

```
docs/
└── testing/
    ├── e2e-scenarios.md
    ├── integration-results.md
    └── known-defects.md
```

Phase 9 — Production Hardening

Goal

Make the system resilient, secure, maintainable, and predictable.

Reliability

Test:

- Network timeout
- Network unavailable
- Server errors
- Retry
- Recovery
- Partial failure
- Corrupt/stale data

Concurrency

Test:

- Duplicate actions
- Double taps
- Simultaneous operations
- Race conditions
- Conflicting updates
- Out-of-order responses

Lifecycle

Test:

- Rotation/configuration changes
- Background/foreground
- Process death
- App restart
- Interrupted operations

Persistence

Test:

- Database integrity
- Migrations
- Stale cache
- Cache invalidation
- Data recovery

Security

Check:

- Authentication
- Authorization
- Secure storage
- Token handling
- Sensitive-data logging
- Network security
- Input validation

Performance

Check:

- Startup time
- Scrolling
- Memory
- Network usage
- Database performance
- Expensive recomposition/work

Also Check

- Accessibility
- Observability
- Compatibility
- Error messages
- Release configuration

Important Rule

Do not add complex infrastructure just to check a “production” box.

For example, build an offline mutation queue only if the actual requirements justify it.

Phase 10 — Final Acceptance & Release

Goal

Prove that the rebuilt system satisfies the original scope.

Final Checklist

Scope

- [ ] All in-scope capabilities implemented.
- [ ] Original success criteria satisfied.
- [ ] Known exclusions documented.

Behavior

- [ ] Critical journeys work end-to-end.
- [ ] Reference behavior has been compared.
- [ ] Important edge cases are handled.

Architecture

- [ ] Boundaries remain clear.
- [ ] No accidental duplicate sources of truth.
- [ ] Dependencies flow in the intended direction.
- [ ] Major hacks have been removed.

Testing

- [ ] Unit tests pass.
- [ ] Integration tests pass.
- [ ] E2E scenarios pass.
- [ ] Failure scenarios verified.
- [ ] Lifecycle scenarios verified.

Security

- [ ] Sensitive data handled correctly.
- [ ] Logs are safe.
- [ ] Authentication/session behavior verified.

Performance

- [ ] Startup acceptable.

- [ ] UI responsive.
- [ ] Memory acceptable.
- [ ] Network/database behavior acceptable.

Release

- [ ] Release build works.
- [ ] Real-device testing completed.
- [ ] Configuration verified.
- [ ] Permissions verified.
- [ ] Privacy considerations documented.
- [ ] Known limitations documented.
- [ ] Final version tagged.

Recommended Project Documentation

```
my-rebuild/
│
├── docs/
│   ├── planning/
│   │   ├── scope.md
│   │   └── success-criteria.md
│   │
│   ├── discovery/
│   │   ├── feature-inventory.md
│   │   ├── user-journeys.md
│   │   ├── observations.md
│   │   └── unknowns.md
│   │
│   ├── requirements/
│   │   ├── business-rules.md
│   │   └── acceptance-criteria.md
│   │
│   ├── design/
│   │   ├── domain-model.md
│   │   ├── state-machines/
│   │   └── data-ownership.md
│   │
│   ├── architecture/
│   │   ├── system-architecture.md
│   │   ├── module-structure.md
│   │   └── adr/
│   │
│   ├── testing/
│   │   ├── test-strategy.md
│   │   └── scenarios.md
│   │
│   └── release/
│       └── readiness.md
│
├── app/
├── core/
├── domain/
├── data/
├── feature/
└── ...
```

Keep documentation lightweight. It exists to preserve decisions and system understanding, not to create paperwork.

Daily Development Workflow

For every work session:

1. Plan

Choose **one specific capability or problem**.

Define:

- What am I building?
- Why now?
- What does it depend on?
- What does “done” mean?

2. Understand

Before coding, confirm:

- Reference behavior
- Requirements
- Domain rules
- State transitions
- Data ownership
- Architecture location

3. Implement

Build the smallest complete piece that fits the existing architecture.

4. Verify

Check:

- Tests
- Acceptance criteria
- Reference behavior
- Failure cases

5. Improve

Before declaring it done:

- Remove unnecessary complexity.
- Fix duplication.
- Fix architectural violations.
- Improve naming.
- Update documentation.
- Record important decisions.

6. Choose Next Task

Move to the next highest-value/risk item.

Recommended Task Board

```
Backlog
   ↓
Ready
   ↓
In Progress
   ↓
Done
```

For each capability track:

```
Capability
├── Requirements
├── Dependencies
├── Implementation
├── Tests
├── Integration
├── Known issues
└── Acceptance criteria
```

Measure progress by **completed capabilities and working journeys**, not number of screens.

Five Rules for Efficiency

1. Timebox Discovery

Do enough exploration to understand the system.

Do not endlessly reverse-engineer invisible internals.

Investigate deeper when an unknown can materially affect your design.

2. Prioritize Dependencies and Risk

Do risky/uncertain problems early.

Do not spend days polishing low-risk screens while the hardest architectural problem remains unknown.

3. Extract Reusable Infrastructure Gradually

Do not create a giant `core` framework on day one.

When a pattern repeatedly appears across features, extract the common infrastructure.

4. Automate Repetition

Use:

- Templates
- Automated tests
- Formatting
- Lint
- CI
- Code generation where appropriate
- Reusable test utilities

Automate repetitive work so your time goes toward system reasoning.

5. Keep Implementation Proportional

Preserve:

- Real business complexity
- Important state
- Failure behavior
- Security
- Concurrency

- Lifecycle behavior
- Performance requirements

Remove:

- Unnecessary abstractions
- Duplicate code
- Speculative infrastructure
- Architecture that exists only for appearance

Ten Rules to Remember

1. **Never start implementation from screens.**
2. **Map workflows before designing architecture.**
3. **Identify entities and ownership before creating modules.**
4. **Model important state machines explicitly.**
5. **Define the source of truth for important data.**
6. **Build vertical slices, not disconnected layers.**
7. **Attack high-risk problems early.**
8. **Treat failure, concurrency, and lifecycle as normal behavior.**
9. **Prefer simple, strong boundaries over elaborate, weak architecture.**
10. **Measure architecture by how cheaply the system can absorb the next feature.**

The Repeatable Loop

For every new capability:

```
OBSERVE
   ↓
MODEL
   ↓
DESIGN
   ↓
IMPLEMENT
   ↓
VERIFY
   ↓
HARDEN
   ↓
REPEAT
```

Final Mental Model

When looking at a reference app, do not ask:

> *“How do I build this screen?”*

Ask:

> *“What system behavior, state, ownership, rules, dependencies, and failure handling must exist for this screen and workflow to work correctly?”*

That question should drive the entire rebuild.

Definition of Done

The project is finished when:

- The in-scope product behavior is implemented.
- Critical user journeys work end-to-end.
- Business rules are explicit.
- State ownership is clear.
- Important failures are handled.
- Lifecycle behavior is tested.
- Concurrency risks are addressed where relevant.
- Security requirements are satisfied.
- Performance is acceptable.
- Architecture is understandable and maintainable.
- Tests provide meaningful confidence.
- The release build works.
- Known limitations are documented.

The goal is **not**:

> *“I recreated all the screens.”*

The goal is:

> *“I built a cohesive system whose behavior is correct, whose boundaries are understandable, and whose next feature can be added without the architecture collapsing.”*